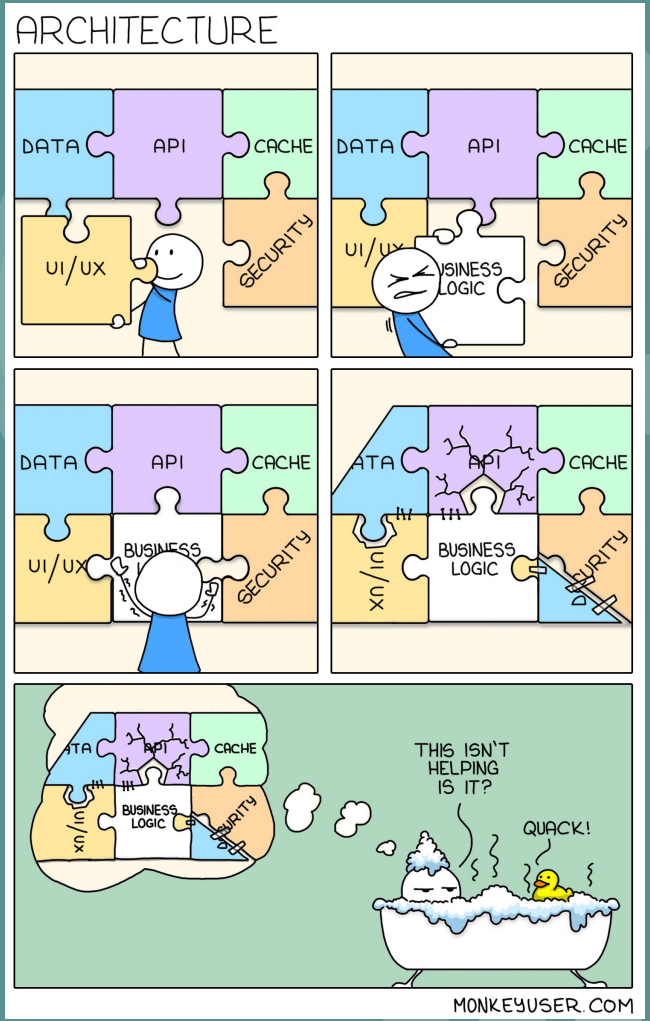
Arquitecturas

En este resumen enumeramos una variedad de Arquitecturas útiles y ampliamente utilizadas.



Clasificación por el tipo dominante de elementos que muestran

Módulos	● Arquitectura en capas
Componentes y conectores (C&C)	● Intermediario (Broker) ● Cliente - Servidor ● Modelo - Vista - Controlador (Model View Controller) ● Tuberías y Filtros (Pipes & Filters) ● De igual a igual (Peer -To -Peer) ● SOA ● Repositorio (Shared Data) ● Multi Nivel (Multi-tier Pattern)
Arquitectura de asignación (Allocation)	● Map-Reduce ● Multi Nivel (Multi-tier Pattern)
Otros	Publicador Suscriptor (publish-subscribe)

Capas

Contexto:
Todos los sistemas complejos experimentan la necesidad de desarrollar y evolucionar partes del sistema de manera independiente. Por esta razón, los desarrolladores del sistema necesitan una separación clara y bien documentada de las preocupaciones, para que los módulos del sistema puedan desarrollarse y mantenerse de manera independiente.

Problema
El software debe segmentarse de tal manera que los módulos
se puede desarrollar y evolucionar por separado con poca interacción entre las partes, soportando portabilidad, modificabilidad y reutilización.

Capas

Solución

Para lograr esta separación de preocupaciones, la arquitectura de capas divide el software en unidades llamadas capas. Cada capa es una agrupación de módulos que ofrece un conjunto cohesivo de servicios.
Debe haber restricciones en la relación de uso permitido entre las capas: las relaciones deben ser unidireccionales.

Restricciones

Cada pieza de software se asigna a exactamente una capa.
Debe haber al menos dos capas (pero generalmente hay tres o Más).
Las relaciones de uso permitido no deben ser circulares .

Gráficamente

Capa de presentación
Capa de lógica de negocios
Capa de persistencia de datos
Capa de base de datos.

Capas

Ejemplos

- Sistemas de comunicación TCP/IP, UDP, NOVEL.
- Sistema Operativo **THE**, construido en Technische Hogeschool Eindhoven en Holanda por E. W. Dijkstra (1968) y sus estudiantes

Cliente - Servidor

Contexto:

Hay recursos y servicios compartidos que una gran cantidad de clientes distribuidos desean acceder, y para los cuales queremos controlar el acceso o la calidad del servicio.

Problema :

Al administrar un conjunto de recursos y servicios compartidos, podemos promover la modificabilidad y la reutilización, eliminando los servicios comunes y teniendo que modificarlos en una única ubicación o en un pequeño número de ubicaciones. Se quiere mejorar la escalabilidad y la disponibilidad mediante la centralización del control de estos recursos y servicios, al tiempo que distribuimos los recursos en varios servidores físicos.

Cliente - Servidor

Solución:

Los clientes interactúan solicitando servicios a el/los servidor(es), que proporcionan un conjunto de servicios. Algunos componentes pueden actuar como **clientes y servidores**. Puede haber un servidor **central** o **varios** distribuidos.
El tipo de conector principal para en la arquitectura cliente-servidor es un conector de datos controlado por un protocolo de solicitud / respuesta (*request/reply*) utilizado para invocar servicios.

Cliente - Servidor

Componentes

Cliente: un componente que invoca servicios de un componente de servidor.
Servidor: un componente que proporciona servicios a los clientes.

Los **servidores** tienen **puertos** que describen los servicios que proporcionan. Las características importantes incluyen información sobre la naturaleza de los puertos del servidor (como la cantidad de clientes que pueden conectarse) y las características de rendimiento (por ejemplo, las tasas máximas de invocación de servicio). Conector de solicitud / respuesta, un conector de datos que emplea un protocolo de solicitud / respuesta, utilizado por un cliente para invocar servicios en un servidor. Las características importantes incluyen si las llamadas son locales o remotas, y si los datos están encriptados.

Cliente - Servidor

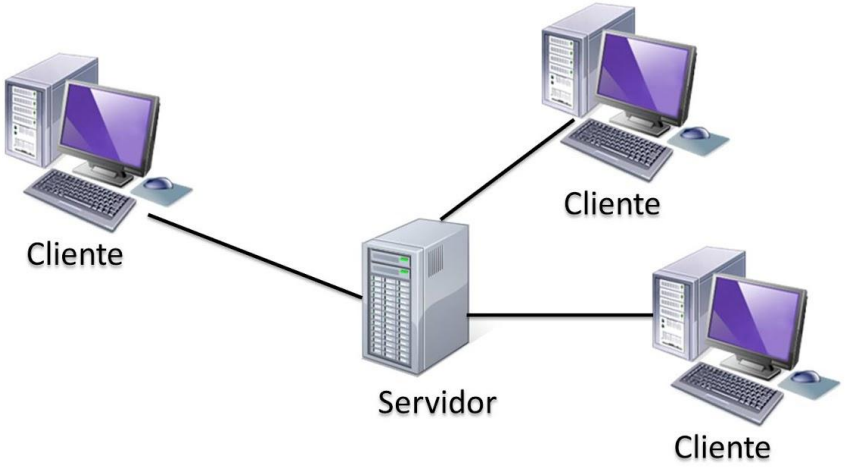
Algunas de las **desventajas** de la arquitectura cliente-servidor son:

- El servidor puede ser un **cuello de botella** en el rendimiento
- Puede ser un **punto único de falla**.
- Además, las decisiones sobre dónde ubicar la funcionalidad a menudo son complejas y costosas de cambiar después de que el sistema se haya puesto en operación.

Cliente - Servidor

Ejemplos
La World Wide Web es el ejemplo más conocido de un sistema que se basa en la arquitectura cliente-servidor, permitiendo a los clientes (navegadores web) acceder a la información de los servidores a través de Internet mediante el Protocolo de transferencia de hipertexto (HTTP).

HTTP es un protocolo de solicitud / respuesta. HTTP es sin estado; la conexión entre el cliente y el servidor finaliza después de cada respuesta del servidor.



Modelo Vista Controlador

Contexto:
El software de **interfaz** de usuario suele ser la parte más frecuentemente modificada de una aplicación interactiva. Por este motivo, es importante mantener las modificaciones del software de la interfaz de usuario separadas del resto del sistema.

Problema:
¿Cómo se puede mantener la funcionalidad de la interfaz de usuario separada de la funcionalidad de la aplicación y aun así responder a las aportaciones de los usuarios o a los cambios en los datos de la aplicación subyacente? ¿Y cómo se pueden crear, mantener y coordinar múltiples vistas de la interfaz de usuario cuando cambian los datos de la aplicación subyacente?

Modelo Vista Controlador

Solución:
La arquitectura modelo-vista-controlador (MVC) separa la funcionalidad de la aplicación en tres tipos de componentes:

- Un modelo, que contiene los datos de la aplicación.
- Una vista, que muestra parte de los datos subyacentes e interactúa con el usuario.
- Un controlador, que media entre el modelo y la vista y administra las notificaciones de cambios de estado.

Debilidad:
La complejidad puede no “valer la pena” para interfaces de usuario simples.

Modelo Vista Controlador

Elementos:

- El **modelo** es una representación de los datos o el estado de la aplicación, y contiene (o proporciona una interfaz para) la lógica de la aplicación.

- La **vista** es un componente de la interfaz de usuario que produce una representación del modelo para el usuario o permite alguna forma de entrada del usuario, o ambos.

- El **controlador** gestiona la interacción entre el modelo y la vista, traduciendo las acciones del usuario en cambios al modelo o cambios en la vista.

Modelo Vista Controlador

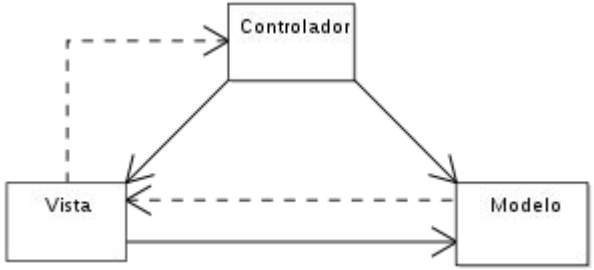
Ejemplos:

La arquitectura MVC se usa ampliamente en bibliotecas de interfaz de usuario como las clases:

- Java's Swing classes.
- Microsoft's ASP.NET framework.
- Adobe's Flex software development kit.
- Nokia's Qt framework.

Por lo tanto, es común que una sola aplicación contenga muchas instancias de **MVC** (a menudo una por objeto de interfaz de usuario).

Gráficamente



Repositorio

Contexto
Varios componentes computacionales necesitan compartir y manipular grandes cantidades de datos. Estos datos no pertenecen únicamente a ninguno de esos componentes.
Problema
¿Cómo pueden los sistemas almacenar y manipular datos persistentes a los que acceden múltiples componentes independientes?

Repositorio

Solución

En la arquitectura de datos compartidos, la interacción está dominada por el intercambio de datos persistentes entre múltiples *accesores* de datos y al menos un almacén de datos compartidos. El intercambio puede ser iniciado por los *accesores* o el almacén de datos. El tipo de conector es lectura y escritura de datos. El modelo computacional general asociado con los sistemas de datos compartidos es que los *accesores* de datos realizan operaciones que requieren datos del almacén de datos y escriben los resultados en uno o más almacenes de datos. Esos datos pueden ser vistos y activados por otros usuarios que acceden a ellos. En un sistema de datos compartidos puros, los *accesores* de datos interactúan sólo a través de uno o más almacenes de datos compartidos.

Repositorio

Elementos

Almacén de datos compartidos. Las preocupaciones incluyen los tipos de datos almacenados, las propiedades orientadas al rendimiento de los datos, la distribución de datos y la cantidad de *accesores* permitido.

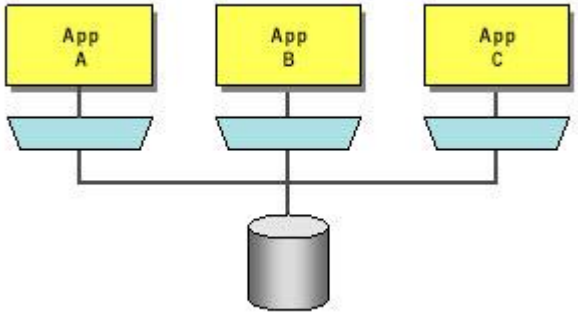
Componente de acceso a los datos. Conector de lectura y escritura de datos. Una opción importante aquí es si el conector es transaccional o no, así como el idioma de lectura / escritura, los protocolos y la semántica.

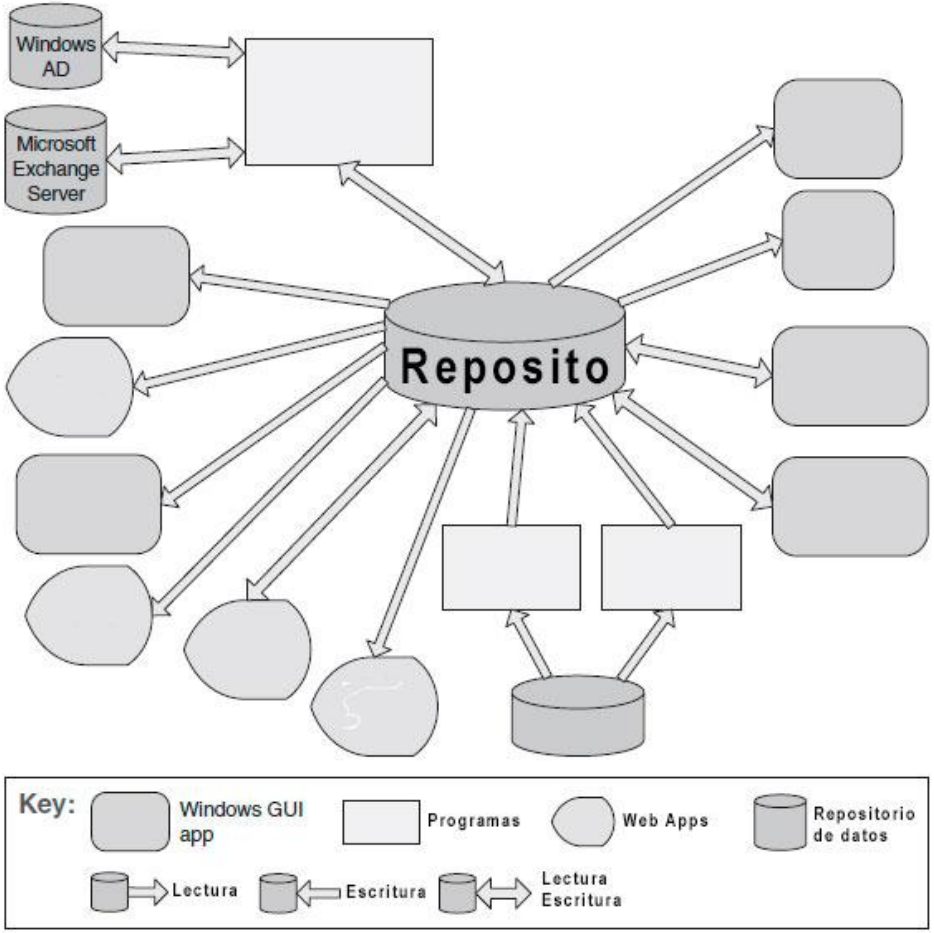
Debilidades

- El almacén de datos compartidos puede ser un cuello de botella de rendimiento.
- El almacén de datos compartidos puede ser un único punto de error.
- Los productores y consumidores de datos pueden estar estrechamente acoplados.

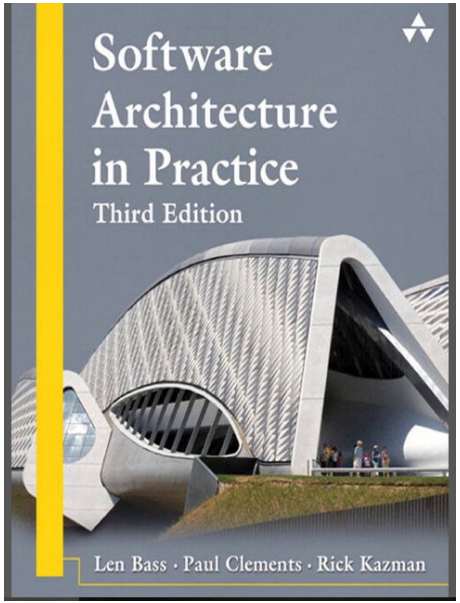
Repositorio

Gráficamente





BIBLIOGRAFÍA

Software Architecture in Practice Third Edition, PEARSON, Len Bass, Paul Clements, Rick Kazman

Capítulo 13 o 14